# Smart Clinic Appointment & Patient Management System (SCAPMS)

**COS202 Programming II — Lab Assessment Project**

Smart Clinic Appointment & Patient Management System (SCAPMS)

C++17 · Qt 6.5.3 · Qt Widgets

**Student:** OlisaEmeka Davies Chinedu-Eze

**Date:** 13 November 2026

**About this document.** The assessment asks for a single document that holds the source code under four headings, the screenshots, a UML diagram and a two to three page report. This file is that document, in the order the brief lists it: **1** the report, **2** the six screenshots, **3** the UML diagrams, **4** the source code, **5** the data files, and **6** the readme and the detailed supplementary report.

**Source code.** The complete source is in the repository https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT. Section 4 of this document lists every file of the program under the four headings the brief names, so the shape of the program can be followed from the page and the code read in the repository.

## Contents


1. Project report
2. Screenshots of the graphical interface
3. UML class diagrams
4. Source code
    1. Header files
    2. Source files
    3. GUI files and resources
    4. Data files required to run the application
5. Readme and the detailed supplementary report

## 1. Project Report

# Academic Project Report

**Course:** COS202 Programming II – Lab Assessment Project **Project:** Smart Clinic Appointment & Patient Management System (SCAPMS) **Student:** OlisaEmeka Davies Chinedu-Eze **Language & Framework:** C++17, Qt 6.5.3 (Qt Widgets) **Build Environment:** Visual Studio 2022 MSVC Build Tools, qmake

---

## 1. Project Description

A clinic running on paper records suffers double bookings, a waiting room nobody can track, and deleted records with no way back. SCAPMS is a desktop application that fixes those problems. It manages patients in three categories — general, emergency and follow-up — and lets a member of staff register, edit and delete a record; book, cancel and complete an appointment; manage a first-in first-out waiting queue; search and sort the records with the algorithm of their choice; undo the last administrative step; and read four clinic reports. All patient and appointment data is written to and read from CSV files, so the clinic survives between runs.

## 2. System Features

| Feature | Where it lives in the interface |
|---|---|
| Register, edit and delete a patient | Patient Records panel |
| Schedule, cancel and complete an appointment | Appointments panel |
| Waiting room: add a patient, call the next one | Waiting Queue panel |
| Search by ID, keyword, name or category; sort by ID, name or age | Search & Sort panel |
| Undo the last change to a patient or an appointment | Administration panel, or Ctrl+Z |
| Save and load the clinic files | Administration panel, or Ctrl+S / Ctrl+O |
| Four clinic reports | Clinic Reports panel |
| Live dashboard and the recently accessed records | Administration panel |

## 3. Class Hierarchy

The core of the model is the **abstract base class** `PatientRecord`, which holds the details every patient has in common: patient ID, name, age, contact number and category. It declares four pure virtual functions — `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()` — so it can never be created on its own. It carries concrete methods that every category shares, and its destructor is virtual, so deleting through a `PatientRecord*` frees the subclass.

**Three subclasses** give each category its own behaviour: `GeneralPatient` (a standard outpatient visit with a consultation fee), `EmergencyPatient` (an urgent case with a severity level 1–3 and a triage note), and `FollowUpPatient` (a returning patient with the date of the last visit and the assigned doctor).

**Interface.** C++ has no `interface` keyword, so **`Appointable`** is a pure abstract class that only declares `scheduleAppointment()`, `cancelAppointment()` (with and without an ID), `hasScheduledAppointment()` and `getAppointmentDetails()`. It holds no data, so any class that can be booked may implement it. `EmergencyPatient` deliberately does not: a trauma case is never booked in advance, and because the database routes every booking through `Appointable`, the rule is enforced by the type system rather than by a check on the category name. A second interface, **`Billable`**, is implemented at the same time through the intermediate abstract class `ScheduledPatient`, showing that a class may implement more than one interface.

**Composition and association.** A patient owns its appointments: `PatientRecord` holds a `std::vector<Appointment>`, so a patient is associated with one or more appointments and they are created and destroyed together. `ClinicDatabase` owns the records in a `std::vector<std::unique_ptr<PatientRecord>>`, so each record is freed exactly once.

**Encapsulation.** Every patient field is private and reached through an accessor. The rules of the clinic — a valid age, a unique ID, a date in the right format — are checked in the controller layer, so the screens and the data can never disagree about them.

## 4. Data Structures Used

| Structure | Where | Why this one |
|---|---|---|
| `std::vector` | the patient roster; the appointments of each patient | O(1) append, and the record list grows and shrinks all day |
| `std::queue` | the consultation waiting room | First-in first-out: the patient who joined first is served first |
| `std::stack` | the administrative undo history | Last-in first-out, exactly what "undo the last step" needs |
| `PatientRecord* recentPatients[10]` | the recently accessed cache | A fixed-size array of ten slots, held oldest at the front and newest at the back. Using a record moves it to the newest place; when it is full the record used longest ago makes room |

## 5. Searching Algorithms Used

Every search is written by hand; the program never calls `std::find` or `std::binary_search`.

1. **Linear search (O(n))** — `linearSearchByID` scans the roster and returns the position of the patient whose ID is *exactly* the text typed, in a list of any order.
2. **Linear keyword search (O(n))** — `linearSearchByText` finds any patient whose ID *or* name holds the text, ignoring letter case, so "GEN" finds every general outpatient.
3. **Binary search (O(log n))** — `binarySearchByID` halves the search space on each step. It needs a sorted list, so the panel makes a sorted copy before calling it, which guarantees the precondition.
4. **Recursive search** — `recursiveSearchByName` calls itself with the next position until it runs off the end of the list, collecting every patient whose name holds the text.
5. **Category search (O(n))** — `searchByCategory` returns every record of one category.

The routine is chosen from a combo box, so the search that runs matches the state of the data: binary search for a sorted list, linear search otherwise.

## 6. Sorting Algorithms Used

Both sorts are written by hand and chosen from a combo box.

1. **Selection sort ($O(n^2)$)** — finds the smallest remaining record and performs one swap per pass, so it does at most n − 1 swaps. It sorts in place and needs no extra memory, which suits the short lists of a clinic.
2. **Merge sort (O(n log n))** — splits the list in half and rejoins it. The merge takes the record of the left half first unless the record of the right half is strictly smaller, which makes the sort **stable**: records with the same key keep the order they had.

Records are ordered by patient ID, patient name or patient age. Names and IDs are compared without regard to letter case, so a name typed in small letters sits in its proper place rather than at the end. A label under the sort controls explains the chosen routine and why it fits, and the sort and the binary search share one comparison rule, so the list stays sorted the way the binary search expects.

## 7. Recursive Algorithm Used

`recursiveSearchByName` walks the records one position at a time. The base case is reached when the position runs off the end of the list; the recursive case tries the next position and joins what it found. The recursion solves a genuine subproblem — the same search on the remainder of the list — rather than being a loop dressed as recursion. Merge sort is recursive in the same way: it splits the list and defers to itself on each half.

## 8. GUI Design

The interface is built in Qt Widgets and split into six tabs, one class per tab: Patient Records, Appointments, Waiting Queue, Search & Sort, Clinic Reports and Administration.

- **Events.** Button clicks, menu actions, combo box changes, text field changes and a one-second timer all go through signals and slots. The File menu holds New, Save as…, Load… and Exit; the Edit menu holds Undo last operation and Quick search.
- **Layouts.** No pixel positioning is used. `QFormLayout` keeps a label with its field, `QHBoxLayout` and `QVBoxLayout` divide the panels into rows and columns, and `QGridLayout` lines up the tables of figures and of category priorities. Using four techniques rather than one means each part of the window is laid out by the one that suits its shape.
- **Advanced techniques.** Rows are colour-coded by category (red for emergency, green for general, blue for follow-up) with a table of categories and priorities beside the roster; the form fields change with the chosen patient type; file dialogs choose the import and export files; the timer keeps the status bar and the dashboard live; every invalid entry raises a message naming the field; shortcuts cover Ctrl+N, Ctrl+S, Ctrl+O, Ctrl+Z, Ctrl+F and Ctrl+Q; and every control carries a tooltip.

## 9. Exception Handling Approach

`ClinicException`, the base of the hierarchy, derives from `std::exception` and overrides `what()`. Four derived types name the failure precisely: `DuplicateIDException` for an ID already registered, `InvalidPatientDataException` for an age outside 0–120 or a blank name, `InvalidAppointmentException` for a booking without a date or a patient that is not registered, and `FileIOException` for a file that cannot be opened, read or written.

Every panel wraps the work it asks the controller to do in `try` / `catch` and shows the message as a dialog, so a mistake never closes the program. There is also a last line of defence: the application class overrides `notify()`, the

one method every event passes through, so an exception that escapes any slot is caught and shown rather than ending the program. Rows in a data file that cannot be read are skipped and listed, and a save writes to a temporary file first, so a failed write cannot destroy the data already on disk.

## 10. Algorithms Chosen and Reasons

- **Two sorts, not one.** Selection sort suits the handful of records a small clinic holds, because it makes the fewest swaps and needs no scratch space; merge sort is there for a large import, because it stays at O(n log n) whatever the order of the data and preserves the order of equal records.
- **Binary search with a defensive copy.** Binary search is only correct on a sorted list, so a sorted copy is made before it is called. The panel works with the records it finds and not with their positions, so the record shown is always the record that was found.
- **One comparison rule everywhere.** The search ignores letter case, so the sort and the binary search do too, which keeps the list sorted the way the binary search expects.
- **One cache order.** The recently accessed records use a single order — oldest at the front, newest at the back — and a cache hit records the access, so the panel really shows the most recently used records.

## 11. Challenges Encountered

1. **One class that did too much.** The panels first called the database directly, so every rule of the clinic ended up in `ClinicDatabase`, which grew to 488 lines and mixed the data handling with the business logic — the very thing the brief forbids.
2. **The binary search showed the wrong patient.** The search returned a position, and the panel used that position in the *unsorted* roster, so the wrong patient appeared.
3. **The cache did not behave like a cache.** Three different ideas of the order were used at once, so the panel showed the records in registration order rather than use order.
4. **A failed save could destroy the data that was already saved.** The file was opened for writing straight away, so an ofstream emptied it before the writing had even begun.
5. **Loading a file threw the appointments away.** The save asked for two files but the load asked for one, so appointments saved under another name were silently lost.
6. **One mistake could end the program.** Qt does not catch an exception that leaves a slot.

## 12. How the Challenges Were Resolved

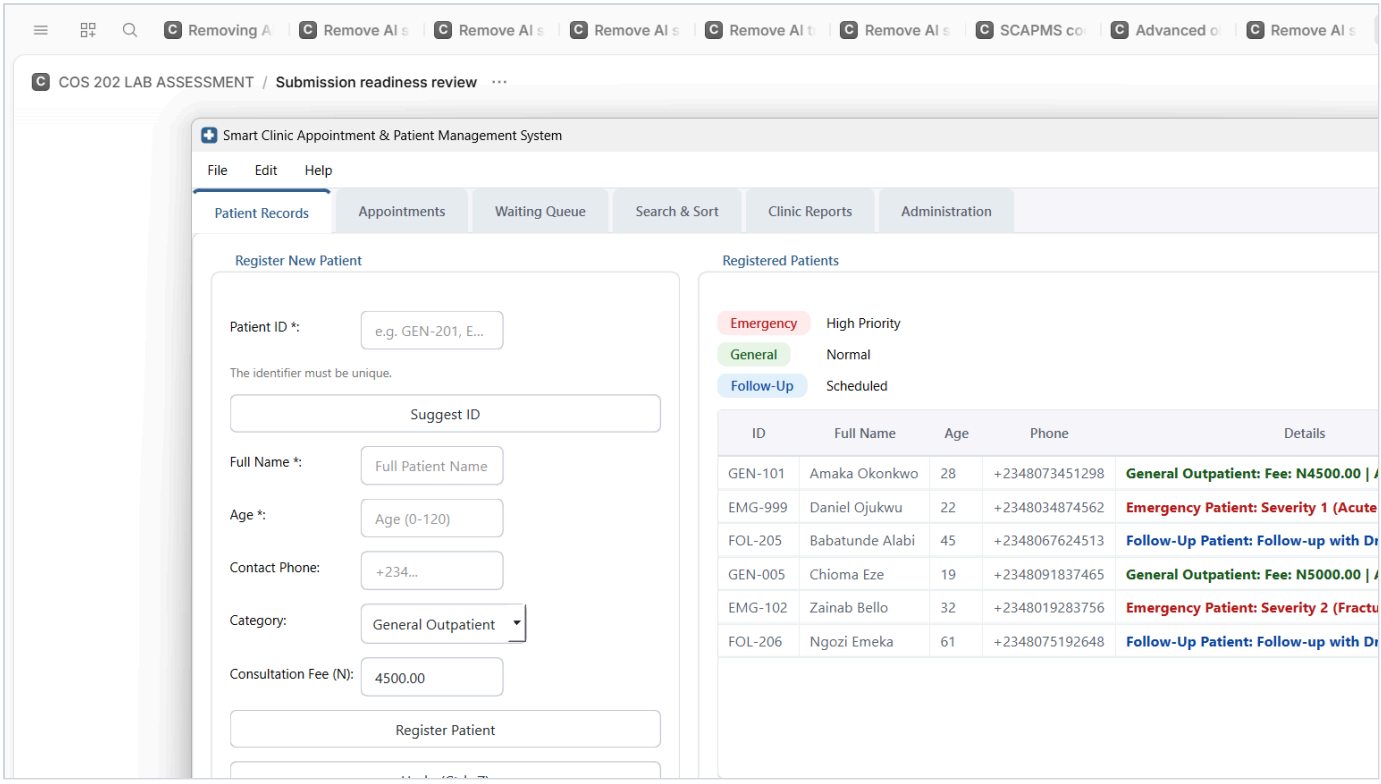
1. The rules were moved up into `ClinicManager` and `AppointmentController` in the controller layer, and the panels were repointed at the manager. `ClinicDatabase` now only stores and finds, and no file in `gui/` refers to it.
2. The search returns the records that matched, not their positions, so the record shown is always the record that was found.
3. One order is used everywhere: a record used again moves to the newest place, a new record is added at the back, and a cache hit records the access.
4. The records are written to a temporary file first, and the real file is only replaced when the writing has finished, so a failed save leaves the old file untouched.
5. The load asks for the appointment file as well, offers the file next to the patient file when the staff cancels, and asks before the clinic is replaced.

6. The application class in `gui_main.cpp` overrides `notify()`, the one place every event passes through, so anything that escapes a slot is caught and shown as a message.

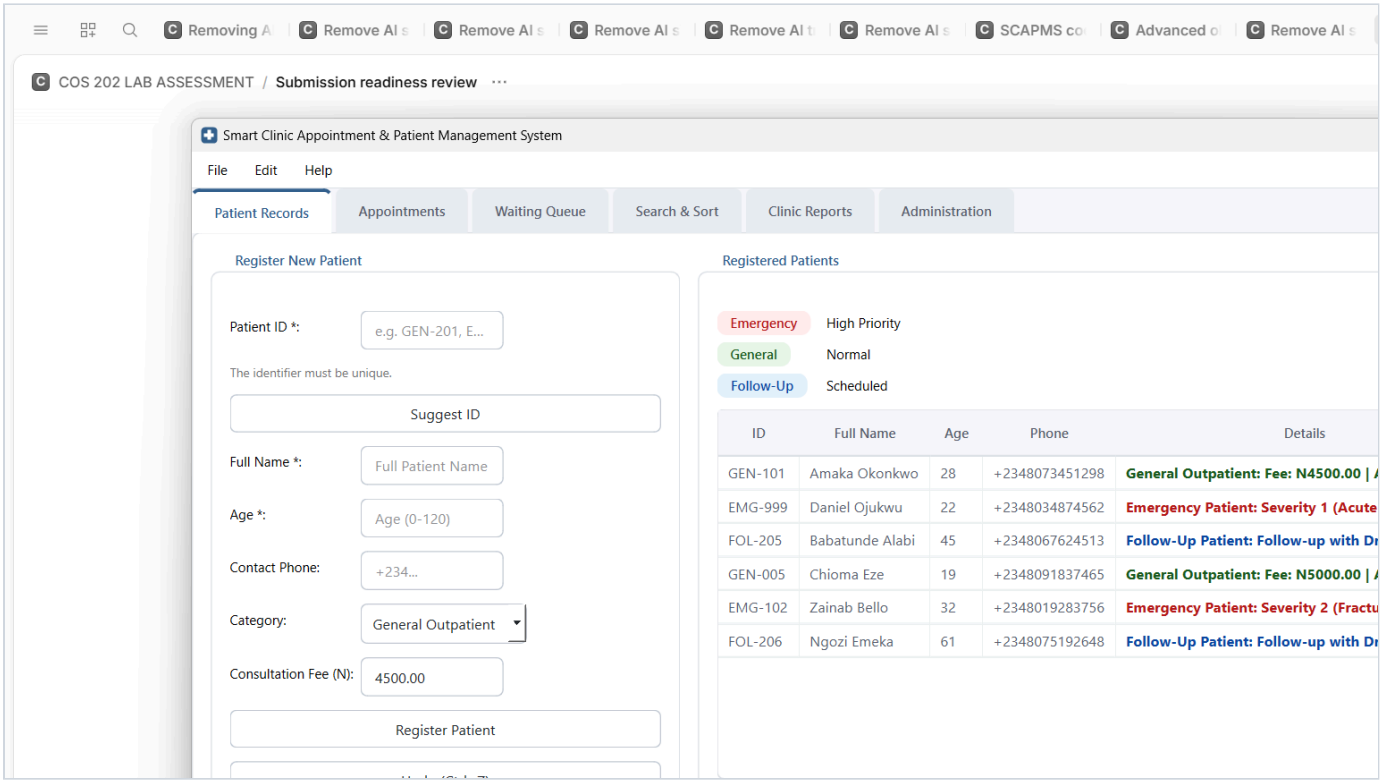
---

*The complete source code is at `https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT` and is also listed in the Source Code section of this document.*

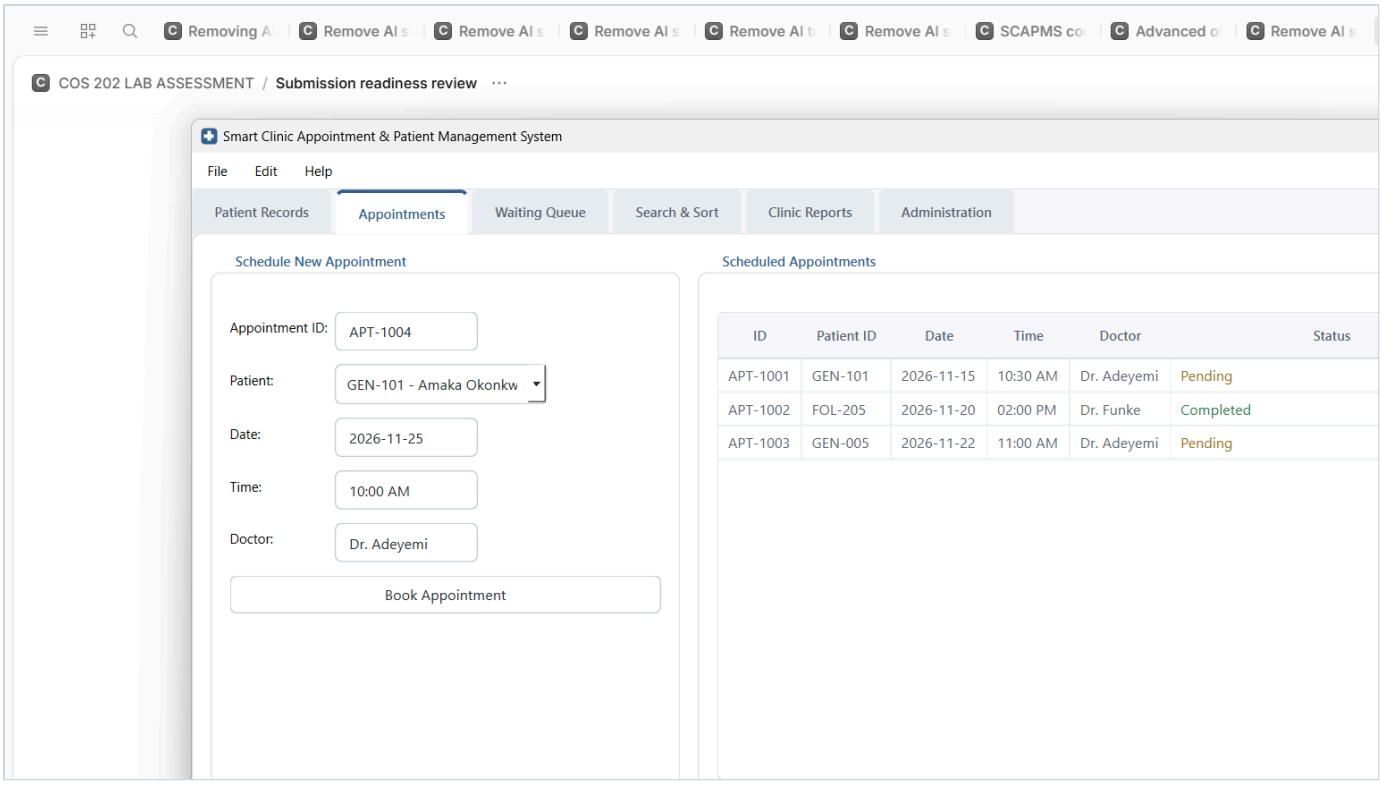
## 2. Screenshots



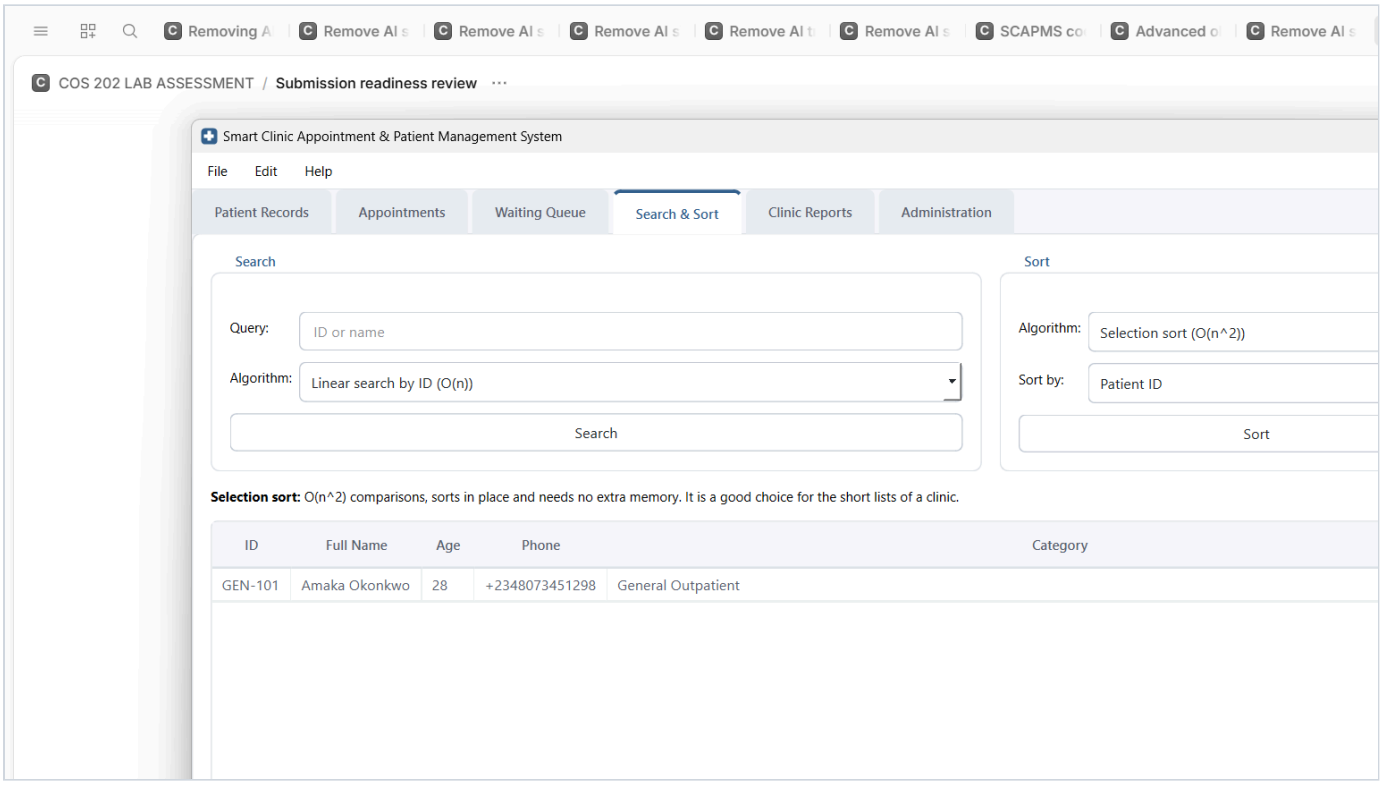

**1_main_dashboard.png** — Main dashboard - the live figures of the clinic, the category priorities and the recently accessed records



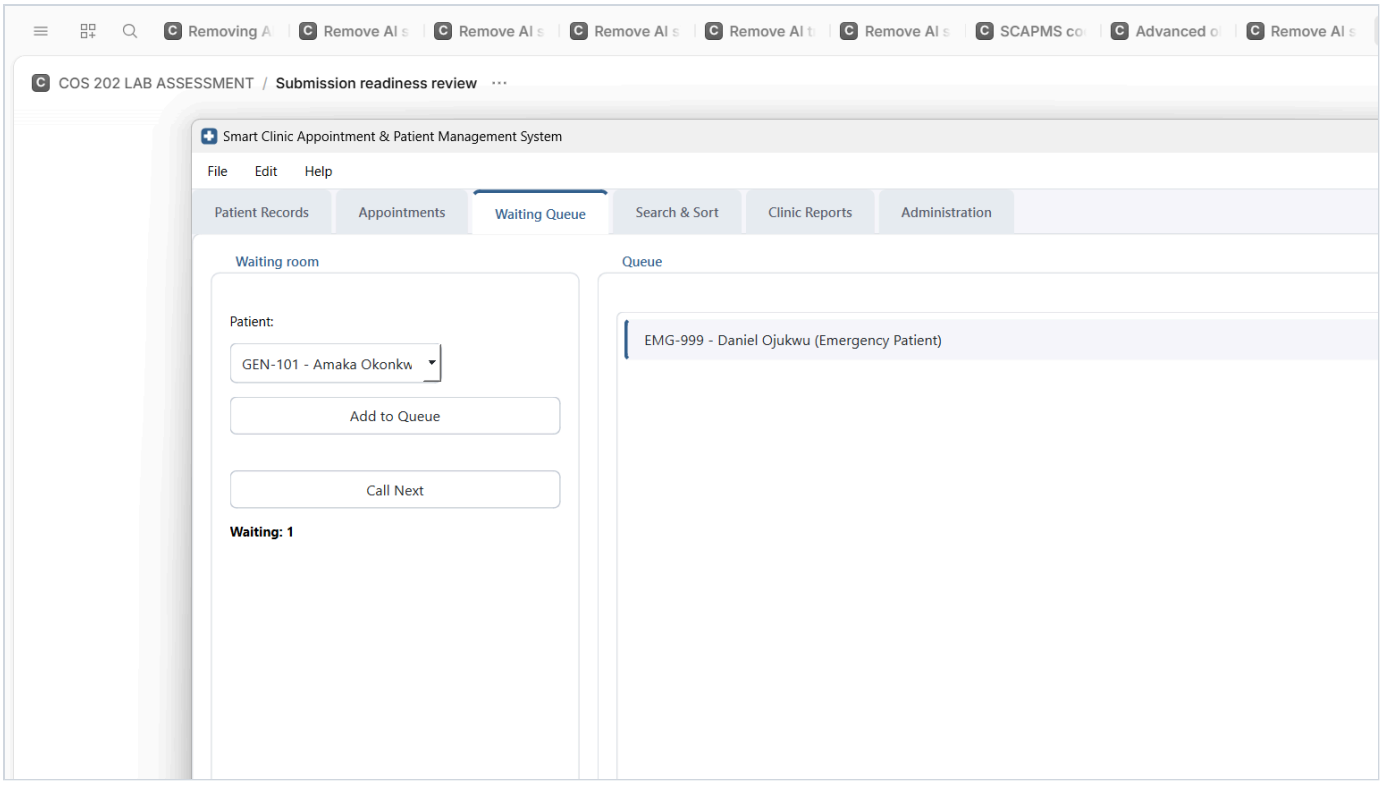

**2_patient_registration.png** — Patient registration - the registration form and the colour-coded roster of the patients



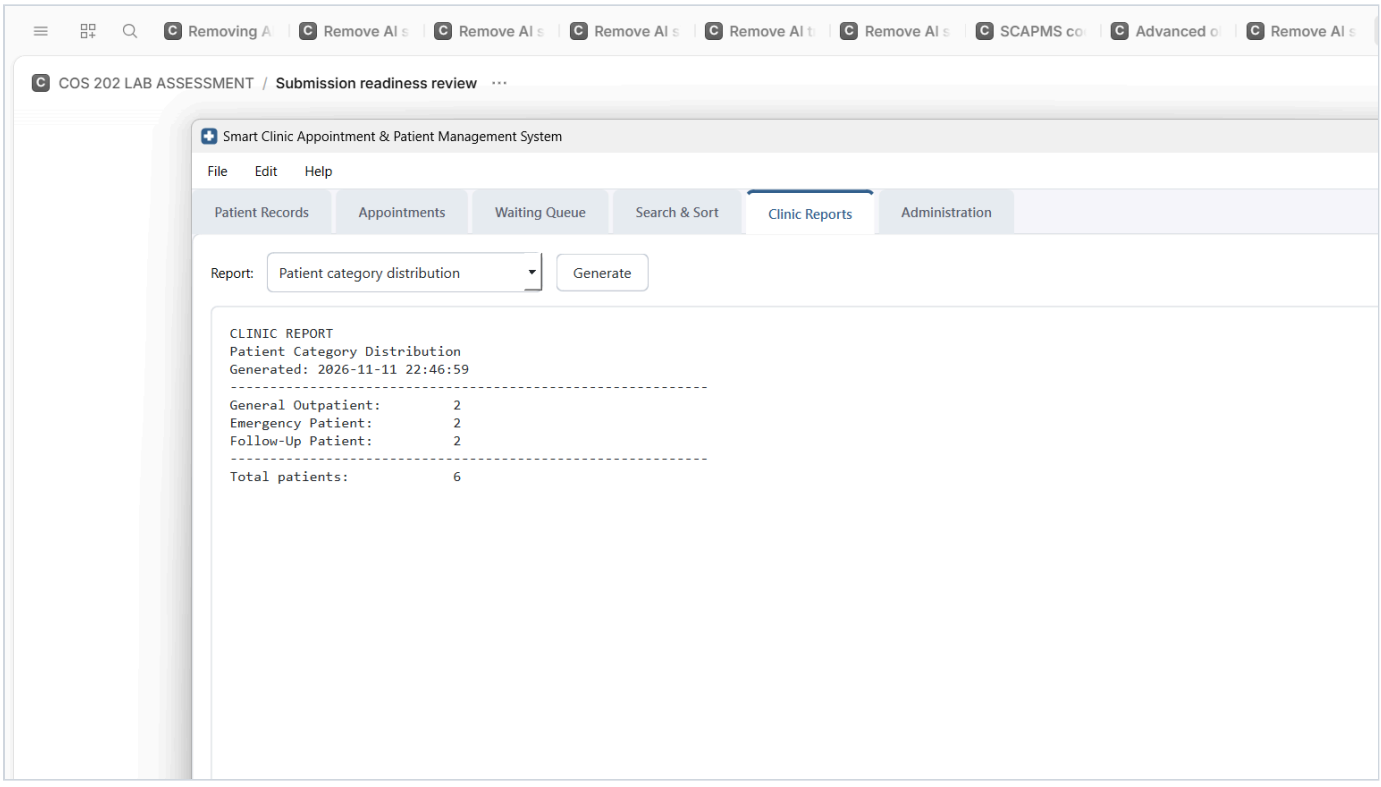

**3_appointment_management.png** — Appointment management - the booking form and the schedule of the clinic




**4_search_and_sort.png** — Search and sort - the algorithm choices and the result of a linear search by patient ID



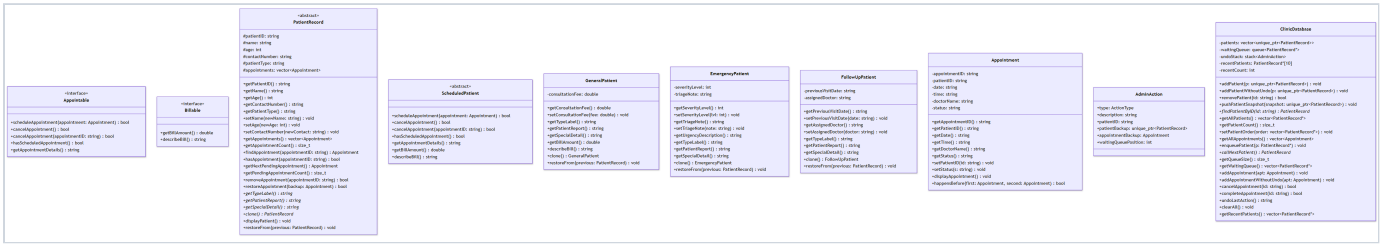

**5_waiting_queue.png** — Waiting queue - a patient in the consultation queue




**6_clinic_reports.png** — Clinic reports - the patient category distribution

## 3. UML Class Diagrams

The full diagram of the program covers twenty five classes and is too tall to read on one page, so it is drawn as the four layers of the program. Each page holds the classes of its own layer, the relationships between them, and the single relationship that joins the layer to the next one, so every arrow of the full diagram is still present. The inheritance arrows, the `«abstract»` and `«interface»` stereotypes, the composition markers and the virtual calls in the base classes show the four things the brief asks a diagram to demonstrate.



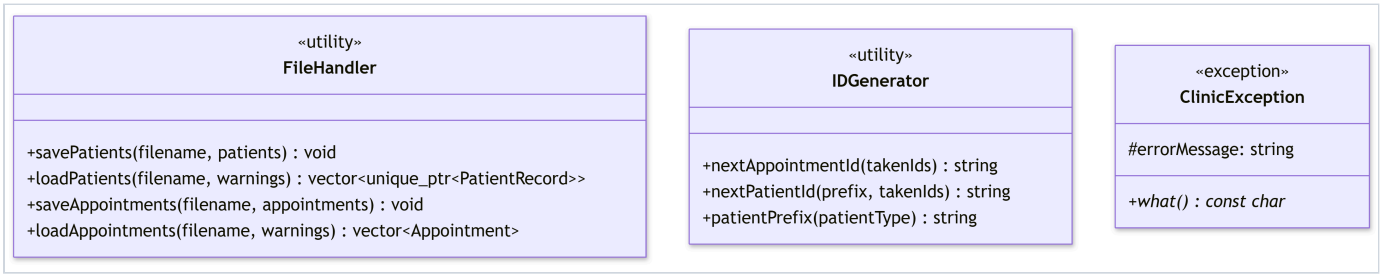
**UML_1_model.png** — Model layer - the abstract base class, the three patient categories, the appointment and the database




**UML_2_controller.png** — Controller layer - the manager, the appointment rules and the two algorithm engines




**UML_3_screens.png** — Interface layer - the main window and one class per tab




**UML_4_utilities.png** — Utility layer - the file handler, the identifier generator and the exception classes

## 4. Source Code

The complete source of the program is in the repository https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT. It is cloned and run like this:

```
git clone https://github.com/OlisaDavies/COS-202-LAB-ASSESSMENT
cd COS-202-LAB-ASSESSMENT
build_gui.bat      builds release\SCAPMS_GUI.exe
build.bat          builds build\SCAPMS.exe and runs the console demonstration
```

The tables below list every file of the program, grouped under the four headings the brief names. Each heading names the folder the files live in, so the repository can be followed file by file.

### 4.1 Header files

Every header of the program, from the model, the controller and the utility layers.

| File |
|---|
| controller/AppointmentController.h |
| controller/ClinicManager.h |
| controller/ReportBuilder.h |
| controller/SearchEngine.h |
| controller/SortEngine.h |
| model/AdminAction.h |
| model/Appointable.h |
| model/Appointment.h |
| model/Billable.h |
| model/ClinicDatabase.h |
| model/EmergencyPatient.h |
| model/FollowUpPatient.h |
| model/GeneralPatient.h |
| model/PatientRecord.h |
| model/ScheduledPatient.h |
| utils/ClinicExceptions.h |
| utils/DateUtil.h |
| utils/FileHandler.h |
| utils/IDGenerator.h |
| utils/StringUtil.h |

### 4.2 Source files

Every source file of the program. `main.cpp` builds the console demonstration and `gui_main.cpp` builds the graphical program; the two cannot be in one binary because each has its own `main()`.

| File |
|---|
| controller/AppointmentController.cpp |
| controller/ClinicManager.cpp |
| controller/ReportBuilder.cpp |
| controller/SearchEngine.cpp |
| controller/SortEngine.cpp |
| model/AdminAction.cpp |
| model/Appointment.cpp |
| model/ClinicDatabase.cpp |
| model/EmergencyPatient.cpp |
| model/FollowUpPatient.cpp |
| model/GeneralPatient.cpp |
| model/PatientRecord.cpp |
| model/ScheduledPatient.cpp |
| utils/DateUtil.cpp |
| utils/FileHandler.cpp |
| utils/IDGenerator.cpp |

### 4.3 GUI files and resources

The entry point of the graphical program, the graphical classes themselves, the look and the build files.

| File |
| --- |
| SCAPMS.pro |
| build.bat |
| build_gui.bat |
| gui/AdminPanel.cpp |
| gui/AdminPanel.h |
| gui/AppointmentPanel.cpp |
| gui/AppointmentPanel.h |
| gui/MainWindow.cpp |
| gui/MainWindow.h |
| gui/PatientPanel.cpp |
| gui/PatientPanel.h |
| gui/QueuePanel.cpp |
| gui/QueuePanel.h |
| gui/ReportPanel.cpp |
| gui/ReportPanel.h |
| gui/SearchPanel.cpp |
| gui/SearchPanel.h |
| gui/clinic.ico |
| gui/resources.qrc |
| gui/style.qss |
| gui_main.cpp |

### 4.4 Data files required to run the application

The two CSV files the import and the export dialogs read and write. The program starts with the same records built into it, so it also runs without them.

| File |
| --- |
| appointments.csv |
| patients.csv |

## 5. Readme and Supplementary Report

### 5.1 Readme

# Smart Clinic Appointment & Patient Management System (SCAPMS)

**Course:** COS202 Programming II - Lab Assessment Project
**Author:** OlisaEmeka Davies Chinedu-Eze
**Technologies:** C++17, Qt 6.5.3 (x64), MSVC 2022

## Project Overview

A desktop application for running an outpatient clinic. Patient records, appointments and the waiting room are all managed from a Qt widgets interface, backed by `std::vector`, `std::queue` and `std::stack`. The search and sort algorithms are written by hand rather than taken from the standard library.

The program is split into four layers, and every layer has its own namespace:

| Layer | Folder | Namespace | What it holds |
|---|---|---|---|
| Model | `model/` | `scapms::model` | The patient classes, the appointment, the database |
| Controller | `controller/` | `scapms::controller` | The searching, sorting and reporting routines |
| GUI | `gui/` | `scapms::gui` | The main window and one panel per tab |
| Utilities | `utils/` | `scapms::utils` | The exception classes, the file reader, the ID generator |

The model, controller and utility layers do not use Qt at all, so the same code runs in the console demonstration ( `main.cpp` ) and in the graphical program ( `gui_main.cpp` ).

## Features

- Patient records for the general, emergency and follow-up categories, with add, edit and delete
- Appointment scheduling and cancellation
- Waiting room queue, served first in first out
- Undo for the last change to a patient or an appointment
- Patient search by linear, binary, recursive and category search, and sorting by selection sort and merge sort
- CSV import and export for patients and appointments
- Reports on category distribution, pending appointments, the daily schedule and frequently attended patients

## Folder layout

```
model/       PatientRecord, Appointable, Billable, ScheduledPatient, GeneralPatient,
             EmergencyPatient, FollowUpPatient, Appointment, AdminAction, ClinicDatabase
controller/  ClinicManager, AppointmentController, SearchEngine, SortEngine, ReportBuilder
```

```
gui/         MainWindow, PatientPanel, AppointmentPanel, QueuePanel,
             SearchPanel, ReportPanel, AdminPanel, style.qss, clinic.ico,
             resources.qrc
utils/       ClinicExceptions, DateUtil, FileHandler, IDGenerator, StringUtil
main.cpp     the console demonstration
gui_main.cpp the entry point of the graphical program
```

## The source code under the headings of the submission

The submission asks for the complete source code under four headings. The files of the project belong to them like this:

### i. Header files

```
model/PatientRecord.h          model/Appointable.h        model/Billable.h
model/ScheduledPatient.h       model/GeneralPatient.h      model/EmergencyPatient.h
model/FollowUpPatient.h        model/Appointment.h         model/AdminAction.h
model/ClinicDatabase.h         controller/ClinicManager.h  controller/AppointmentController.h
controller/SearchEngine.h      controller/SortEngine.h     controller/ReportBuilder.h
utils/ClinicExceptions.h       utils/DateUtil.h            utils/FileHandler.h
utils/IDGenerator.h            utils/StringUtil.h          gui/MainWindow.h
gui/PatientPanel.h             gui/AppointmentPanel.h      gui/QueuePanel.h
gui/SearchPanel.h              gui/ReportPanel.h           gui/AdminPanel.h
```

### ii. Source files

```
model/PatientRecord.cpp        model/ScheduledPatient.cpp  model/GeneralPatient.cpp
model/EmergencyPatient.cpp     model/FollowUpPatient.cpp   model/Appointment.cpp
model/AdminAction.cpp          model/ClinicDatabase.cpp    controller/ClinicManager.cpp
controller/AppointmentController.cpp                       controller/SearchEngine.cpp
controller/SortEngine.cpp      controller/ReportBuilder.cpp utils/DateUtil.cpp
utils/FileHandler.cpp          utils/IDGenerator.cpp       main.cpp
```

### iii. GUI files and resources

```
gui/MainWindow.cpp              gui/PatientPanel.cpp        gui/AppointmentPanel.cpp
gui/QueuePanel.cpp              gui/SearchPanel.cpp         gui/ReportPanel.cpp
gui/AdminPanel.cpp              gui_main.cpp                gui/style.qss
gui/clinic.ico                  gui/resources.qrc           SCAPMS.pro
build_gui.bat                   build.bat                  .vscode/tasks.json
.vscode/launch.json
```

### iv. Data files

```
patients.csv                    appointments.csv
```

`patients.csv` and `appointments.csv` hold the records that the import and the export dialogs read and write. The program starts with the same records built into it, so it also runs without them.

The program has two entry points: `gui_main.cpp` builds the graphical program and `main.cpp` builds the console demonstration. They cannot be in one binary because each of them has its own `main()`, so the console version is built by `build.bat` with the compiler directly and is not part of `SCAPMS.pro`.

## Opening a panel from the command line

The panels of the graphical program can also be asked for on the command line, which is how the screenshots in the submission were produced and which makes a demonstration of any single panel possible without clicking:

```
SCAPMS_GUI.exe                    opens the first panel
SCAPMS_GUI.exe 3                  opens the Waiting Queue
SCAPMS_GUI.exe 3 GEN-101          also puts that patient in the queue
SCAPMS_GUI.exe 2 GEN-101 0        also runs a search for GEN-101 with the
                                  linear search by ID
```

The number is the position of the tab, in the order the tabs are shown: 0 Patient Records, 1 Appointments, 2 Waiting Queue, 3 Search & Sort, 4 Clinic Reports, 5 Administration. The second and the third words only mean something for

the two panels they belong to, so an argument that does not fit the panel is ignored.

## The submission document

`SCAPMS_Submission.pdf` is the single document the assessment asks for. It holds, in the order the brief lists them: the two to three page report, the six screenshots, the UML class diagrams, and the complete source code under the four headings the brief names. `SUBMISSION_REPORT.md` is the report on its own, and the four `UML_*.png` pictures are the diagrams, because a full diagram of Twenty five classes do not read on one page when they are printed.

## Where each requirement is implemented

| Requirement | Implementation |
|---|---|
| 1.a Abstract base class | `model/PatientRecord.h` - `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()` are pure virtual; `displayPatient()` and `restoreFrom()` are concrete methods shared by every category |
| 1.b Subclasses | `model/GeneralPatient.h`, `model/EmergencyPatient.h`, `model/FollowUpPatient.h` |
| 1.c Interface | `model/Appointable.h` - `scheduleAppointment()`, `cancelAppointment()` and the overloaded `cancelAppointment(id)`; `model/Billable.h` is a second interface, so a class can implement more than one |
| 1.d Runtime polymorphism | `main.cpp` walks the records through a `PatientRecord*` and calls `displayPatient()`, which dispatches to the version of the category; `gui/PatientPanel.cpp` shows the same report through the base pointer |
| 1.e Composition | `model/PatientRecord.h` owns the `std::vector<Appointment> appointments` collection, so a patient holds one or more appointments; `model/ClinicDatabase.h` owns the patient records |
| 1.f / 2 Organisation | The four layers above, each with its own namespace; every class with behaviour is split into a header and a source file |
| 2 Separation of responsibilities | `controller/ClinicManager.h` is the only class the GUI talks to. `controller/AppointmentController.h` holds the rules of a booking. `model/ClinicDatabase.h` only stores and finds. The GUI never calls the database directly. |
| 3.a Vector | `model/PatientRecord.h` - `std::vector<Appointment> appointments`; `model/ClinicDatabase.h` - `std::vector<std::unique_ptr<PatientRecord>> patients` |
| 3.b Queue | `model/ClinicDatabase.h` - `std::queue<PatientRecord*> waitingQueue`. The patient who joins first is served first |
| 3.c Stack | `model/ClinicDatabase.h` - `std::stack<AdminAction> undoStack`, with `model/AdminAction.h` holding the copy of the record that is put back. Undoing a delete also gives the record its place in the waiting queue again |
| 3.d Fixed-size array | `model/ClinicDatabase.h` - `PatientRecord* recentPatients[10]`, shown in the Administration panel. The array is held from the oldest record at the front to the newest at the back, the most recently used record is shown first, and when the cache is full the record that was used the longest time ago makes room for a new one |
| 4 Searching | `controller/SearchEngine.h` - linear by ID or name, binary by ID on sorted records, recursive by name, linear by category. The text is matched without regard to letter case, and a part of the identifier or the name is enough |
| 5 Sorting | `controller/SortEngine.h` - selection sort and merge sort, chosen from a combo box on the Search & Sort panel. The chosen order is kept by the controller, so the roster and the results show the same order. The names and the identifiers are compared without regard to letter case |
| 6 Events | `gui/MainWindow.cpp` and the panels - button clicks, menu actions, combo box changes, text field changes and a `QTimer` that refreshes the status bar and the dashboard. The File menu holds New, Save as, Load and Exit |
| 7 Panels | Patient Records, Appointments, Waiting Queue, Search & Sort, Clinic Reports, Administration |
| 8 Layout | `QFormLayout` for the forms, `QHBoxLayout` and `QVBoxLayout` for the panels, `QGridLayout` for the tables of the categories and their priorities |

| Requirement | Implementation |
| --- | --- |
| 9 Advanced GUI | 9.a colour coded rows with a table of the categories and their priorities, 9.b fields that change with the patient type, 9.c file dialogs, 9.d status bar timer, 9.e validation messages, 9.f shortcuts, 9.g tooltips |
| 10 Exceptions | `utils/ClinicExceptions.h` - `DuplicateIDException`, `InvalidPatientDataException`, `InvalidAppointmentException`, `FileIOException`. `gui_main.cpp` also catches anything that escapes a slot, so that the program never stops by itself |
| 11 Persistence | `utils/FileHandler.h` - CSV read and write for both patients and appointments. A file is written to a temporary name first, so a failed save cannot destroy the data that is already on the disk |

## The rules of the clinic

The rules live in the controller layer, so that the screens and the data always agree on them.

| Rule | Where it is checked |
| --- | --- |
| The age has to be between 0 and 120 | `ClinicManager::checkPatientRules` |
| The name may not be empty | `ClinicManager::checkPatientRules` |
| The identifier has to begin with the prefix of the category: `GEN-`, `EMG-` or `FOL-` | `ClinicManager::checkPatientRules` |
| A contact number that is given has to look like a telephone number | `ClinicManager::checkPatientRules` |
| The same patient may not wait in the queue twice | `ClinicManager::enqueuePatient` |
| The date has to be written as `YYYY-MM-DD` and the time as `HH:MM AM` or `HH:MM PM` | `AppointmentController::checkBookingRules` |
| The same doctor cannot be booked twice in the same slot | `AppointmentController::scheduleAppointment` |
| An emergency patient is not booked in advance, it goes to the waiting room | `AppointmentController::scheduleAppointment` |
| A visit that was completed cannot be cancelled, and a cancelled visit cannot be completed | `ScheduledPatient::cancelAppointment` |

## Screenshots

The `screenshots/` folder holds the six views of the graphical program, in the order in which the submission asks for them:

| File | View |
| --- | --- |
| `1_main_dashboard.png` | The dashboard of the clinic, with the live figures and the categories |
| `2_patient_registration.png` | The registration form and the roster of the patients |
| `3_appointment_management.png` | The booking form and the schedule of the clinic |
| `4_search_and_sort.png` | The search and the sort, with the result of a search |
| `5_waiting_queue.png` | The waiting room and the patients that are waiting |
| `6_clinic_reports.png` | The report of the clinic |

## Build & Run

### Prerequisites

- Windows 10/11 (x64)
- Visual Studio 2022 C++ Build Tools
- Qt 6.5+

### Compilation

Run the build script for the graphical program:

```
build_gui.bat
```

The binary is generated at `release\SCAPMS_GUI.exe`.

`build.bat` builds the console version (`build\SCAPMS.exe`), which walks through the exception handling, the polymorphic calls, the searching and sorting routines, the waiting room, the undo stack, the summary of the clinic and the four reports without needing the GUI.

### Data files

`patients.csv` and `appointments.csv` hold the same records the application starts with, and are the files the import and export dialogs read and write. The program starts with the same records built into it, so it also runs when these two files are not there.

### 5.2 Detailed supplementary report

The report above follows the twelve items the brief lists. This fuller account keeps the record of every problem met while the program was built, and is included as supporting material.

# Academic Project Report

**Course:** COS202 Programming II
**Project Title:** Smart Clinic Appointment & Patient Management System (SCAPMS)
**Author / Student:** OlisaEmeka Davies Chinedu-Eze
**Language & Framework:** C++17, Qt 6.5.3 (x64)
**IDE & Build Environment:** Visual Studio 2022 MSVC Build Tools / Visual Studio Code

---

## 1. Project Overview & Problem Statement

A clinic still running on paper or a spreadsheet ends up with double bookings, a waiting room nobody can keep track of, deleted records with no way back, and no way to see how the day is going. This project fixes those problems with a desktop application.

The **Smart Clinic Appointment & Patient Management System (SCAPMS)** is a desktop application for running an outpatient clinic. It handles: 1. Patient demographic registration and categorization. 2. Advance consultation scheduling and status tracking. 3. First-In, First-Out (FIFO) consultation waiting queue coordination. 4. Fast manual searching and multi-attribute record sorting. 5. Administrative transaction history with instant rollback (Undo). 6. File persistence via structured Comma-Separated Values (CSV). 7. Diagnostic reporting of clinic attendance trends.

## 2. Object-Oriented System Architecture

The code is split into four layers - Model, Controller, GUI and Utilities - and each layer has its own C++ namespace (`scapms::model`, `scapms::controller`, `scapms::gui`, `scapms::utils`). Every class that has behaviour is written as a header file with the declarations and a source file with the implementation; the two pure interfaces and the small inline helper headers are header only by design. The Model, Controller and Utility layers do not depend on Qt, so the same code also runs in the console demonstration.

### 2.0 Separation of Responsibilities (Requirement 2)

The program is built as three layers that only ever depend on the layer below them:

```
GUI layer          PatientPanel, AppointmentPanel, QueuePanel,
   |                SearchPanel, ReportPanel, AdminPanel, MainWindow
   |  asks the controller to do the work
   v
Controller layer    ClinicManager, AppointmentController,
   |                SearchEngine, SortEngine, ReportBuilder
   |  applies the rules of the clinic, then stores the result
   v
Model layer         PatientRecord and its subclasses, Appointment,
                    AdminAction, ClinicDatabase
```

The rule that makes the separation real is that **the screens never call the database directly**. Every panel holds a `ClinicManager&` and asks it to do the work, in the same way that the console demonstration does. Before this restructure the panels called `ClinicDatabase` themselves, which meant the business rules and the data handling had ended up in one class of 488 lines. The rules now live in the controller layer:

- `ClinicManager` is the single point of contact for the GUI. It owns the database and the appointment controller, and it applies the rules of a patient record (the age has to be between 0 and 120, the name may not be empty, the identifier may only be used once).
- `AppointmentController` holds the rules of a booking (the date and the time have to be written in the format of the clinic, the patient has to be registered, and the category of the patient has to be one that can be booked in advance).
- `ClinicDatabase` now only stores and finds. The few exceptions it still raises are the ones that follow from the shape of the data itself, such as an identifier that is already in use.

The classes named in the brief are all present in their own layer: `ClinicManager`, `SearchEngine` and `AppointmentController` in the Controller, `MainWindow`, `PatientPanel`, `AppointmentPanel`, `AdminPanel` and `SearchPanel` in the GUI, `IDGenerator` and `FileHandler` in the Utilities, and the patient classes and `Appointment` in the Model.

The `AdminPanel` is a sixth tab that holds the operations that belong to the person in charge of the clinic rather than to the treatment of a patient: undoing the last step, saving and loading the data, and starting a new day with an empty database. It also shows the live dashboard of the clinic and the records that were used most recently, which are the ten records held in the fixed size cache. The dashboard is built out of real widgets rather than out of one block of text: a table of the six figures of the clinic, and a table of the three categories with the priority that goes with each of them.

## 2.1 Abstract Base Class & Inheritance Hierarchy

At the core of the clinical model is the abstract base class `PatientRecord` (`model/PatientRecord.h`): - **Encapsulated Attributes:** `patientID`, `name`, `age`, `contactNumber` and `patientType`. - **Pure Virtual Functions (`= 0`):** `getTypeLabel()`, `getPatientReport()`, `getSpecialDetail()` and `clone()`. Each derived class has to provide its own version of them. - **Concrete Methods:** `displayPatient()` and `restoreFrom()` are implemented in the base class and shared by every category. `displayPatient()` writes the report that the category produced, so it is also the method that the console demonstration calls through a base-class pointer. - **Virtual Destructor:** Ensures safe polymorphic deallocation through base pointers without resource leakage.

Three specialized subclasses inherit from `PatientRecord`: 1. `GeneralPatient`: Represents standard outpatient visits. Encapsulates a `consultationFee` attribute, which is quoted in naira. 2. `EmergencyPatient`: Models urgent trauma admissions. Encapsulates an urgency `severityLevel` (1-3) and a qualitative `triageNote`. 3. `FollowUpPatient`: Facilitates continuity of care. Encapsulates a `previousVisitDate` and `assignedDoctor`.

## 2.2 Interface Design (`Appointable` and `Billable`)

C++ does not possess a native `interface` keyword. Consequently, abstract interfaces were constructed as pure abstract classes possessing solely pure virtual functions and a virtual destructor:

```
class Appointable {
public:
    virtual ~Appointable() = default;
    virtual bool scheduleAppointment(const model::Appointment& appointment) = 0;
    virtual bool cancelAppointment() = 0;
    virtual bool cancelAppointment(const std::string& appointmentID) = 0;
    virtual bool hasScheduledAppointment() const = 0;
    virtual std::string getAppointmentDetails() const = 0;
};
```

`Appointable` also demonstrates **compile-time polymorphism** through method overloading: the two `cancelAppointment()` methods share a name and differ only in their parameter list, so the choice between cancelling the next booking and cancelling a named one is made by the compiler.

A second interface, `Billable` (`model/Billable.h`), declares `getBillAmount()` and `describeBill()`. The intermediate abstract class `ScheduledPatient` inherits from the base class **and from both interfaces at the same time**, which demonstrates that a class may implement more than one interface:

```
class ScheduledPatient : public PatientRecord, public Appointable, public Billable { ... };
```

`GeneralPatient` and `FollowUpPatient` therefore answer both interfaces, while `GeneralPatient` overrides the billing methods with its consultation fee. The shared booking code is written once in `ScheduledPatient`, so the two subclasses only add their own category fields.

By clinical logic, `EmergencyPatient` deliberately omits `Appointable`, reflecting hospital protocol where trauma cases cannot be scheduled in advance. Because the database routes every booking through the `Appointable` interface, this rule is enforced by the type system rather than by a check on the category name: an attempt to book an emergency case is rejected with a `InvalidAppointmentException`, and the emergency case reaches the doctor through the waiting room instead.

### 2.3 Polymorphism & Composition

- **Runtime Polymorphism:** All patient records are managed through base-class pointers (`PatientRecord*`). Virtual function dispatch dynamically routes calls to derived subclass implementations via internal virtual method tables (VTables). The console demonstration calls `displayPatient()` through a `PatientRecord*`, and the graphical panel builds the same report through `getPatientReport()`, so the polymorphic call is exercised by both programs.
- **Composition ("HAS-A"):** A patient owns its appointments. `PatientRecord` holds a `std::vector<Appointment> appointments` collection, so a patient is associated with one or more appointments and the appointments are created and destroyed together with their owner. The `ClinicDatabase` owns the patient records in a `std::vector<std::unique_ptr<PatientRecord>>` and gathers the appointments of all patients on demand for the schedule table and the reports.
- **Ownership:** The database owns the live records and deletes them in its destructor. The undo stack owns one *copy* of the record for each administrative action, so the stack and the roster never share the same memory. The undo entry holds the copy in a `std::unique_ptr`, which makes it movable but not copyable, and frees the copy automatically when the entry is destroyed.

## 3. Data Structures & Justification

| Data Structure | C++ Implementation | Complexity | Clinical Justification in SCAPMS |
|---|---|---|---|
| **Vector** | `std::vector<std::unique_ptr<PatientRecord>>` | O(1) amortized append | Manages the dynamic list of registered patients. The unique pointers keep the ownership of each record explicit, so a record is freed exactly once. |
| **Vector** | `std::vector<Appointment>` | O(1) amortized append | The appointments a patient holds. The patient owns them, so they are created and destroyed together with the record. |
| **Queue** | `std::queue<PatientRecord*>` | O(1) push / pop | Enforces First-In, First-Out (FIFO) triage order in the consultation waiting room. |
| **Stack** | `std::stack<AdminAction>` | O(1) push / pop | Powers the administrative **Undo** engine (LIFO). Accidental record deletions or modifications are popped and reversed in reverse chronological order. |
| **Array** | `PatientRecord* recentPatients[10]` | O(1) indexed access | A fixed-size 10-slot cache storing the Most Recently Accessed records, reducing database scan frequency for active patients. The array is held from the oldest record at the front to the newest at the back; a record that is used again moves to the newest place, and when the cache is full the record that was used the longest time ago is the one that is dropped. |

## 4. Manual Algorithm Implementation

The brief rules out the STL algorithms, so every search and sort below is written by hand:

### 4.1 Searching Algorithms

1. **Linear Search (O(N)):** Implemented in `SearchEngine::linearSearchByText`. Scans arbitrary, unsorted rosters element-by-element. It matches the identifier **and** the name, without regard to letter case, and a part of either is enough - so the staff can type "chi" and find "Chioma Eze". It returns **every** match, not only the first one.
2. **Binary Search (O(log N)):** Implemented in `SearchEngine::binarySearchByID`. Operates on sorted rosters by calculating the midpoint `mid = low + (high - low) / 2` and halving the search space each cycle. The GUI makes a sorted copy of the records before calling it, so the precondition always holds.
3. **Recursive Search:** Implemented in `SearchEngine::recursiveSearchByName`. Solves patient lookup by recursively advancing an index pointer until hitting either base case (match located or end of array reached). It collects every patient whose name holds the text.
4. **Category Search:** Implemented in `SearchEngine::searchByCategory`. Returns every record of a category, which is what the clinic staff needs when they want to see, for example, all emergency cases.

### 4.2 Sorting Algorithms

1. **Selection Sort (O(N^2)):** Iteratively identifies the minimum element in the unsorted partition and performs a single swap per outer iteration. Highly suitable for small datasets where write operations must be minimized.
2. **Merge Sort (O(N log N)):** A divide-and-conquer algorithm that recursively bisects the patient vector into singletons and merges them into sorted temporary buffers. Guarantees optimal O(N log N) execution regardless of initial data distribution. The merge takes the record of the left half first when both records have the same key, so the sort is **stable** and equal records keep the order they had.
3. **Multi-Field Comparator:** Sorting algorithms accept an abstract criteria parameter ( `SortField::PatientID` , `SortField::PatientName` , `SortField::PatientAge` ), permitting dynamic user-driven reordering from the GUI.
4. **Case-Insensitive Comparison:** The name and the identifier are compared without regard to letter case, using the same rule as the search. A name that was typed in small letters therefore sits in its proper place in the list instead of being pushed to the end by the ASCII order, and the binary search keeps working, because it uses the very same comparison that the sort uses.

## 5. Graphical User Interface (GUI) & Event Architecture

The graphical frontend was built natively using **Qt 6.5.3 (C++17)**: - **Layout Management (Req. 8):** Hardcoded pixel coordinates were strictly avoided. Responsive layouts adapt dynamically to window resizing using `QVBoxLayout` , `QHBoxLayout` , `QFormLayout` and `QGridLayout` . The forms use `QFormLayout` , so a label and its field stay together and the form grows with its longest label; the panels use `QHBoxLayout` and `QVBoxLayout` , so the columns and the rows share the space that is left; and the three tables of figures and of categories and priorities use `QGridLayout` , so the entries line up under each other in both directions. - **Event-Driven Programming (Req. 6):** User actions trigger Qt Signals connected to C++ Member Slots. - *Button events (6.a):* Add, Edit, Delete, Undo, Book, Mark as Completed, Cancel, Search, Sort, Save, Load and New Database are all wired to a slot. - *Menu events (6.b):* The File menu holds **New**, **Save as...** ( `Ctrl+S` ), **Load...** ( `Ctrl+O` ) and **Exit** ( `Ctrl+Q` ); the Edit menu holds **Undo last operation** ( `Ctrl+Z` ) and **Quick search** ( `Ctrl+F` ); the Help menu holds **About**. - *Text field events (6.c):* the appointment identifier field records that the staff typed it, so that the free identifier is not written over their work; and the patient identifier field answers while it is being typed, telling the staff whether the identifier is free or already in the clinic. - *Timer events (6.d):* a one-second `QTimer` keeps the clock and the live numbers of the clinic in the status bar, shows a reminder of the bookings that are still pending, and refreshes the statistics of the dashboard. - **Multi-Panel Navigation (Req. 7):** Organized via a central `QTabWidget` containing: - *Panel 1:* Patient Registration form and color-coded Roster table ( `gui/PatientPanel.cpp` ). - *Panel 2:* Appointment booking form and schedule grid ( `gui/AppointmentPanel.cpp` ). - *Panel 3:* Waiting room line manager with FIFO calling dialogs ( `gui/QueuePanel.cpp` ). - *Panel 4:* Search and sort engine with real-time algorithm selection dropdowns ( `gui/SearchPanel.cpp` ). - *Panel 5:* Clinic Reports generator ( `gui/ReportPanel.cpp` ). - *Panel 6:* Administration panel with the administrative steps, the live dashboard of the clinic and the recently accessed records ( `gui/AdminPanel.cpp` ). - **Separation of concerns (Req. 2):** Every tab is its own panel class that owns its widgets and talks to the `ClinicManager` , never to the database. `gui/MainWindow.cpp` only builds the tabs, the menu bar and the status bar, and it refreshes the panels when one of them reports a change. - **Advanced GUI Techniques (Req. 9):** - Custom table cell presentation using soft colors ( `#ffebee` for Emergency, `#e8f5e9` for General, `#e3f2fd` for Follow-Up), with a table of the categories and their priorities beside the roster and beside the dashboard. - Fields that appear and disappear with the selected patient type. - Background `QTimer` maintaining live clock synchronization, queue size updates, a reminder of the pending appointments and a live refresh of the dashboard statistics. - Standard file dialog integration via `QFileDialog` for custom file import/export. - Accessible keyboard shortcuts ( `Ctrl+N` , `Ctrl+S` , `Ctrl+O` , `Ctrl+Z` , `Ctrl+F` , `Ctrl+Q` ). - A live hint under the patient identifier field that says whether the identifier can be used. - A tooltip on every button, every field and every list, which says what the control does in one short sentence.

## 6. Exception Handling

A custom exception class hierarchy inheriting from standard `std::exception` guarantees error tolerance without abrupt application termination: - `ClinicException`: Base exception class providing overridden `what() const noexcept`. - `DuplicateIDException`: Thrown when attempting to register an ID already present in the database. - `InvalidPatientDataException`: Thrown for illegal ages (< 0 or > 120) or blank mandatory name fields. - `InvalidAppointmentException`: Thrown for an appointment without an ID, patient, date or time, for a duplicated appointment ID and for a patient that is not registered. - `FileIOException`: Thrown during file stream access failures.

All GUI event slots wrap database mutations inside `try { ... } catch (const ClinicException& e) { ... }` blocks, presenting descriptive `QMessageBox::critical` alerts to users. Rows in a data file that cannot be read are skipped and listed for the user instead of silently turning into empty records.

**The safety net.** Qt does not catch an exception that leaves a slot: it travels out of the event loop and ends the program with "*terminate called after throwing an instance of ...*". Because requirement 11 asks for a program that keeps running after a normal mistake, `gui_main.cpp` uses a small `QApplication` subclass whose `notify()` method catches anything that escapes an event and shows it to the staff as a message. The rules of the clinic are checked in the controller layer, so the panels usually never let an exception through, but the program still has a last line of defence.

**The rules are checked on every way in.** A record is checked when it is registered, when it is edited and when it is read from a file. The contact number rule, for example, was at first only applied on registration, so an edit could put an illegal contact number into a record. The edit path now runs the same `ClinicManager::validatePatient` check as the registration path.

## 7. Challenges and Fixes

1. **One Class That Did Too Much:**
2. *Challenge:* The graphical panels called the database directly, so every rule of the clinic - a valid age, a date written correctly, a patient that may be booked - ended up inside `ClinicDatabase`, which grew to 488 lines and mixed the data handling with the business logic.
3. *Resolution:* The rules were moved up into `ClinicManager` and `AppointmentController` in the controller layer, and the panels were repointed from the database to the manager. `ClinicDatabase` now only stores and finds.
4. **The Binary Search Showed The Wrong Patient:**
5. *Challenge:* The panel made a sorted copy of the records and searched in that copy. The routine returned the **place** of the record it found, and the panel then used that place to look the record up in the unsorted roster, so it showed a patient that had nothing to do with the search.
6. *Resolution:* The search returns the records that matched, not their positions, so the record shown is always the record that was found.
7. **The Cache Of Frequently Accessed Records Did Not Work:**
8. *Challenge:* The fixed-size array of ten records was written with three different ideas of the order at the same time, so it kept the order in which the records were *registered* rather than the order in which they were *used*. A cache hit did not even record the access.
9. *Resolution:* One order is used everywhere now: the array is held from the oldest record at the front to the newest at the back, using a record moves it to the newest place, and a new record is added at the back.
10. **A Failed Save Could Destroy The Data That Was Already Saved:**

11. *Challenge:* The file was opened for writing straight away, and an ofstream empties the file the moment it is opened. If the writing then failed, a half written file was left behind and the program still said "The data was written".
12. *Resolution:* The records are written to a temporary file first, and the real file is only replaced when the writing has finished, so a failed save leaves the old file untouched.
13. **Loading A File Could Silently Throw The Appointments Away:**
14. *Challenge:* The save asked for two file names, but the load asked for one and then looked for a file called "appointments.csv" next to the patient file. Appointments saved under any other name were silently lost, and loading emptied the whole clinic without asking.
15. *Resolution:* The load asks for the appointment file as well, offers the one next to the patient file when the staff cancels, and asks before the clinic is replaced.
16. **One Mistake Could End The Whole Program:**
17. *Challenge:* Qt does not catch an exception that leaves a slot. The exception travels out of the event loop and the program ends with "terminate called after throwing an instance of ...", and the windows disappear.
18. *Resolution:* The application class in gui_main.cpp overrides `notify()`, the one place every event passes through, so anything that escapes a slot is caught and shown as a message.

## 8. Conclusion

The Smart Clinic Appointment & Patient Management System is a working C++ application that covers the lab requirements: object-oriented design, manual algorithms, the required data structures, and a Qt interface.